

National University of Computer and Emerging Sciences, Lahore Campus

	Course Name:	Operating Systems	Course Code:	CS2006
	Degree Program:	BSCS	Semester:	Fall 2022
	Exam Duration:	60 Minutes	Total Marks:	40
	Paper Date:	28-Sep-2022	Weight	15%
	Section:	ALL	Page(s):	6
	Exam Type:	Mid 1 Exam		

Student : Name: _____ Roll No. _____ Section: _____

Instruction/Notes: Attempt all questions on this sheet. Avoid cutting and blotting this paper.
Use last page as rough sheet.

	Q-1	Q-2	Q-3	Q-4	Total
Total Marks	10	10	10	10	40
Marks Obtained					

Question 01: Consider three processes: Parent, Child, and Grandchild. The parent passes a message to the child and the child passed another message to the grandchild. The Parent should wait until both the children are successfully executed. Given the following code skeleton, you are required to complete the missing portion of the code. Note that you will use only ONE *pipe* to complete the communication. (10 points) [CLO-2]

```
#include<stdio.h>
#include<unistd.h>

int main() {
    int pipefd[2];
    int returnstatus1;
    char pipe1writemessage[20] = "Hi child";
    char pipe1writemessage2[20] = "Hi grandchild";
    int pid;
    char readmessage[40];

    if (pipe(pipefd) == -1) {
        printf("Unable to create pipe 1 \n");
        return 1;
    }

    pid = fork();
    if (pid != 0) {
        printf("In Parent: Writing to pipe - Message is %s\n", pipe1writemessage);
        //write message here; close unnecessary file descriptors if needed

        close(pipefd[0]);
        write(pipefd[1],pipe1writemessage,strlen(pipe1writemessage1));
    }
}
```

```

        wait(NULL);

        close(pipefd[1]);

    } else if (pid == 0) { //child process
        printf("This is child \n");
        //read and print message here; close unnecessary file descriptors if needed

        read(pipefd[0],readmessage,strlen(readmessage));
        write(1,readmessage,strlen(readmessage));
        close(pipefd[0]);


        printf("Creating grand child\n");
        int pid2 = fork();
        if (pid2 == 0) { //grand child
            printf("This is grand child\n");
            //read and print message here; close unnecessary file descriptors if needed

            close(pipefd[1]);
            read(pipefd[0],readmessage,20);
            write(1,readmessage,strlen(readmessage));
            close(pipefd[0]);


            return 0;
        } else if (pid2 > 0) {
            printf("In Child: Writing to pipe - Message is %s\n", pipe1writemessage2);
            //write message here; close unnecessary file descriptors if needed
            close(pipefd[0]);
            write(pipefd[1],pipe1writemessage2,strlen(pipe1writemessage2));
            wait(NULL);
            close(pipefd[1]);


            return 0;
        }
    }

    return 0;
}

```

Question 02: (10 points) [CLO-1 CLO-2]

```
#include <sys/types.h>
#include<iostream>
#include<unistd.h>
using namespace std;

#define SIZE=5

int nums[SIZE] = {0,1,2,3,4};

int main() {
    int i;
    pid_t pid;
    pid = fork();
    if (pid == 0)
    {
        for(i=0; i<SIZE; i++) {
            nums[i]*=-i;
            cout<< "CHILD: "<<nums[i]<<endl; // LINE X
        }
    } else if (pid > 0) {
        wait(NULL);
        for(i=0; i<SIZE; i++)
            cout<< "Parent: "<<nums[i]<<endl; // LINE Y
    }

    return 0;
}
```

What output will be at **Line X** and **Line Y**?

Output:

```
CHILD: 0 CHILD: -1 CHILD: -4 CHILD: -9 CHILD: -16
PARENT: 0 PARENT: 1 PARENT: 2 PARENT: 3
PARENT: 4
```

Question 03: What is the context-switch? Explain it with the help of a diagram. Why is context-switch considered as overhead? (10 points) [CLO-1 CLO-3]

Answer:

Question 04: (10 points)

- A) Write a C program that forks a child process that ultimately becomes a zombie process. This zombie process must remain in the system for at least 10 seconds. (4 points) [CLO-1 CLO-2]

Answer:

B) If a parent did not invoke `wait()` and instead terminated, thereby leaving its child processes as orphans. Traditional UNIX systems addressed this scenario by assigning the `init` process as the new parent to orphan processes. The `init` process periodically invokes `wait()`, thereby allowing the exit status of any orphaned process to be collected and releasing the orphan's process identifier and process-table entry.

- B) Why do we need **init** process? (2 points)

Answer:

- C) Answer **YES/NO** and provide a brief explanation. (4 points)

- (a) Can two processes be concurrently executing the same program executable?

Answer:

(a) Yes, two processes can run the same program.

- (b) Can two running processes share the complete process image in physical memory (not just parts of it)?

Answer:

(b) No, in general. (Only time this is possible is with copy-on-write during fork, and before any writes have been made.)

```
int main()
{
    pid_t retVal;
    retVal=fork();
    if(retVal<0)
    {
        perror("fork");
        exit(1);
    }else if(retVal==0)
    { //Child Process
        exit(0);
    }else
    { //Parent Process
        sleep(10);
        int status;
        wait(&status);
        exit(0);
    }
}
```

